

Jaroku — Onboarding, Authentication & First-Run

Complete build spec: Google OAuth, magic link, Tauri deep-link, and per-user account onboarding

Audience: the engineer (or Claude Code) implementing this. **Depends on:** Tauri wrap (in progress — this spec assumes desktop app shell exists with deep-link protocol registered and Node backend running as sidecar), provider-agnostic OIDC verification (v0.2.6), workspace/RLS/tenancy (v0.2.5), `audit_log`, `SecretStore`, and the subscription/billing spec (subscription state per workspace, Free tier as default).

This spec covers, in order:

1. Two distinct onboarding types and their triggers
2. First-run flow (device-level, Tauri app initialization)
3. Authentication: Google OAuth + email magic link
4. Deep-link callback handling (Tauri-specific)
5. Account onboarding (per-user, cross-device)
6. Data model
7. Backend endpoints
8. Email delivery
9. Resume-from-interruption logic
0. Edge cases and failure modes
1. Build order and acceptance criteria

Nothing in this spec is decorative. Every field, every screen, every check is production-necessary. If a step looks skippable, it's because a real failure mode motivates it — read the reasoning.

1. Two onboarding types

These are conceptually separate events that happen in sequence, driven by separate triggers, tracked by separate flags. Conflating them produces bugs where re-installing the app re-runs account onboarding (annoying) or signing out doesn't re-run the auth flow (broken).

1.1 First-run onboarding (device-level)

Trigger: app launch when a local marker file is missing (`~/ .jaroku/app-initialized` on Unix, `%APPDATA%\jaroku\app-initialized` on Windows).

Purpose: ensure the Tauri app has everything it needs to actually run — Python runtime, checkpoint db initialized, filesystem permissions granted.

Runs once per device install. Re-runs only if the user explicitly wipes local state (or a fresh install).

1.2 Account onboarding (user-level)

Trigger: authenticated session where `users.onboarding_completed_at` IS NULL.

Purpose: get the user from "signed in" to "has something running." Workspace named, provider key set (or explicit skip), first agent generated (or explicit skip), landed in the app.

Runs once per user, cross-device. A user signing in on their second device sees their onboarded state (workspace, keys, agents all present) — not the onboarding flow again.

1.3 Trigger matrix

Scenario	First-run	Account
Fresh install, first user ever	✓	✓
Sign out, sign back in (same user, same device)	✗	✗
Same user, second device	✓ (new device)	✗ (flag is user-scoped, server-side)
Sign out, sign in as different user, same device	✗	✓ (new user)
App update (auto-updater)	✗	✗ (use "what's new" popup, not onboarding)
User manually clears <code>~/ .jaroku/</code>	✓	✗ (server-side flag persists)

2. First-run flow (Tauri, device-level)

Runs before any sign-in attempt. Purely local; no network required until step 3.

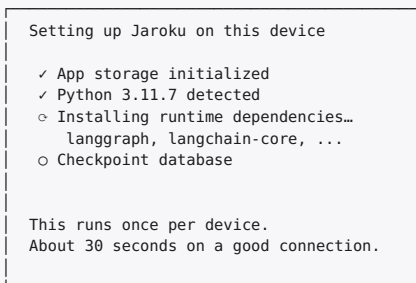
2.1 Screens

Screen 1: Welcome



Nothing to configure here. Single "Get started" button. This screen exists because launching straight into a technical dependency check reads as hostile.

Screen 2: Runtime check

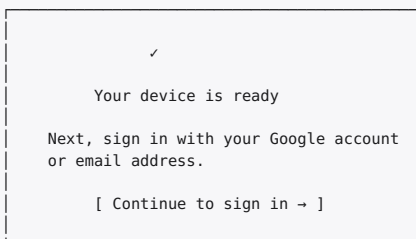


Reuses the design system's amber-in-flight treatment with `stream-pulse` for the current step. Static ✓ for complete, static ○ for pending. Consistent with every other in-flight indicator in the app.

Steps in order:

1. **App storage initialized** — create `~/.jaroku/` directory tree, write initial config file, set permissions. Cannot fail unless the disk is unwritable (surface as a hard error with a "try a different location" option).
2. **Python runtime detection** — check for Python 3.11+ on PATH. Two paths depending on the Tauri wrap decision: - **If Python is bundled** with the Tauri app (option 1 from the Tauri wrap discussion): this step verifies the bundled Python starts correctly, then moves on. - **If Python is externally required** (option 2): this step detects installed Python. If missing or wrong version, offers to install via `uv` (recommended) or shows OS-specific instructions with a "Recheck" button.
3. **Runtime dependency installation** — `uv pip install` the pinned dependency set (from `runtime/pyproject.toml`) into a Jaroku-owned virtual environment at `~/.jaroku/venv`. Never touch the user's system Python or their virtualenvs. Streaming progress from `uv`'s stdout, line by line.
4. **Checkpoint database initialization** — create the SQLite checkpoint database at `~/.jaroku/checkpoints/`, run LangGraph's checkpointer setup (this is the same code path as `JAROKU_CHECKPOINTER` from the Unreleased Storage Isolation session, just applied locally rather than to a hosted Postgres).

Screen 3: Ready



On click: write `~/.jaroku/app-initialized` marker file, transition to authentication (§3).

2.2 Offline handling

If the runtime dependency install requires network (it does — `uv` pulls from PyPI) and the network is unavailable:

- Steps 1, 2, and 4 complete offline (they don't need network).
- Step 3 fails cleanly with a "You're offline" screen, a retry button, and no scary error messages.
- The marker file is NOT written on incomplete first-run. User must complete first-run to proceed.

2.3 Failure and recovery

Any step that fails displays a clear error with three actions: **Retry**, **Get help** (links to a documented troubleshooting page), and **Quit**. No silent failures, no "restart the app" as the only remedy.

If a user quits mid-first-run, they land back at Screen 1 on next launch. First-run must be atomic: either fully complete (marker written) or fully retry-able from the start.

3. Authentication

Two providers, both leading to the same authenticated session state:

- **Google OAuth** — primary CTA, standard OIDC via Google
- **Email magic link** — secondary, custom implementation

3.1 Sign-in screen

Both buttons **appear equally valid**. Google is primary in visual weight because it's the majority path, but "Continue with email" is not buried as a "more options" link — that's the pattern that quietly kills the alternative path for legitimate users who prefer it.

The Terms/Privacy line at the bottom is legally required before the user takes any action that creates an account. Not a checkbox — "By continuing" is standard, enforceable consent for a low-stakes SaaS. Save the checkboxes for actually optional things.

No "sign up" vs "sign in" distinction. Same field, same button, same flow. The server figures out whether to create a new user or authenticate an existing one. This is universally better UX than making people remember which state they're in.

3.2 Google OAuth flow

Uses the existing OIDC infrastructure (v0.2.6). Google is just another OIDC provider — provider-agnostic JWKS verification is already there.

Sequence:

1. User clicks "Continue with Google" in the Tauri app.
2. Tauri app opens the **system browser** (not an embedded webview) to Google's OAuth authorization endpoint. - `redirect_uri`: `https://auth.jaroku.dev/oauth/google/callback` (a Jaroku-owned web endpoint — not a `localhost` redirect, not `jaroku://` directly) - `scope`: `openid email profile` (no more — do not request Gmail access, Drive access, or anything else; requesting more scope than needed triggers Google's scary consent screen) - `state`: a signed, single-use token generated by Jaroku backend that identifies this specific sign-in attempt. Bound to the Tauri app instance via a locally-generated nonce delivered through the deep-link at step 5. - `code_challenge` + `code_challenge_method=S256`: PKCE is mandatory — without it, any process that can intercept the redirect can steal the authorization code.
3. User completes Google's consent flow.
4. Google redirects to `https://auth.jaroku.dev/oauth/google/callback?code=...&state=...`
5. The web endpoint: - Verifies `state` (single-use, unexpired, signed by us) - Exchanges `code` for tokens via Google's token endpoint (using PKCE verifier) - Verifies the ID token (JWKS, signature, `iss`, `aud`, `exp`, `nonce`) — the exact same verification path already

established in v0.2.6 - Provisions the user in the database if new (`users` row, `email_verified: true`, `name` from Google's claim), or looks up existing user - Generates a short-lived **session ticket** — a signed, single-use, 60-second-TTL token - Redirects the browser to `jaroku://auth/complete?ticket=<ticket>`

- The Tauri app receives the deep-link (§4), calls `POST /v1/auth/session` with the ticket, and receives a real session.
- The web endpoint additionally shows a "You can close this tab" confirmation page — the redirect happens automatically, but the confirmation is a fallback if the deep-link handler doesn't fire (browser blocks it, OS asks for confirmation, etc.).

Why a web callback and not `localhost` or `jaroku://` directly:

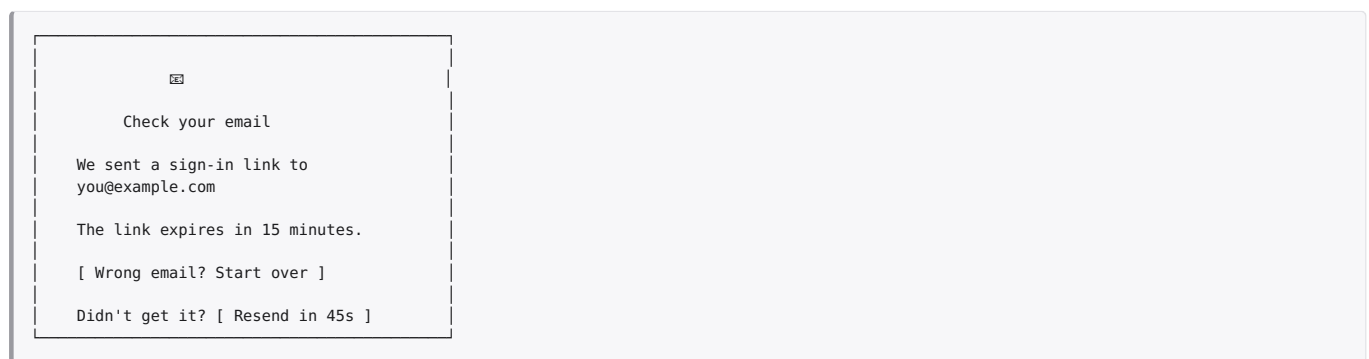
- Google requires HTTPS redirect URIs; `localhost` works only in dev, `jaroku://` isn't accepted at all
- A stable, Jaroku-owned callback URL means Google client configuration doesn't change per user or per install
- The web callback is stateless (it just exchanges the code and hands off) — no session state stored there, minimizing attack surface

3.3 Email magic link flow

New code path (OIDC infrastructure doesn't cover this — magic link is not OIDC). Well-known pattern; implementation is straightforward but must be done right.

Sequence:

- User enters email address in the Tauri app and clicks "Continue with email."
- Tauri app calls `POST /v1/auth/magic-link` with `{ email }`.
- Backend: - Validates email format (a real regex, or a library — do not roll your own weakly) - Rate-limits: max 3 requests per email per hour, max 10 per IP per hour. Prevents abuse (someone spamming a target's inbox by repeatedly submitting their email). - Generates a signed, single-use, 15-minute-TTL magic link token. Stored server-side keyed by hash of the token (never store the raw token — a database dump would then be a mass-account-takeover vector). - Sends an email (§8) with a link: `https://auth.jaroku.dev/magic?token=<token>&email=<email>` - **Always returns 200** to the client, regardless of whether the email exists in the database or not. Do not distinguish "sent" vs "user not found" — this prevents email enumeration attacks. The user always sees "Check your email for a link."
- Tauri app shows a confirmation screen:



- User opens their email, clicks the magic link.
- Browser opens `https://auth.jaroku.dev/magic?token=...&email=...`
- Web endpoint: - Verifies token (signed, unexpired, not already consumed, matches stored hash for this email) - **Marks the token consumed atomically** (a `UPDATE ... WHERE consumed_at IS NULL RETURNING *` in one query) — prevents double-use if the user clicks the link twice quickly - Provisions the user if new (`users` row, `email_verified: true`, `name: NULL` — will be collected in the post-verification screen §3.4) - Generates a session ticket (same 60-second, single-use pattern as OAuth) - Redirects to `jaroku://auth/complete?ticket=<ticket>`
- Tauri app receives deep-link, exchanges ticket for session — same as OAuth path from step 6.

Key security properties of the magic link:

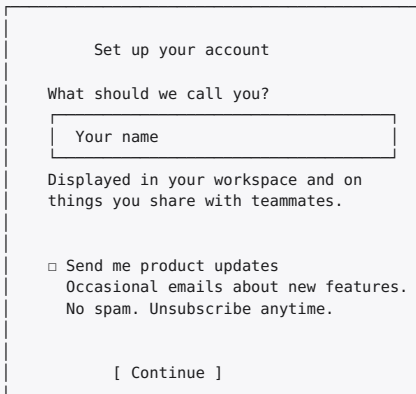
- Signed (HMAC or JWT with a strong secret) — prevents forgery
- Hash stored in database, not raw token — prevents DB-leak → account takeover
- Single-use, atomic consumption — prevents double-use and race conditions
- 15-minute expiry — small enough that a stolen email → account takeover window is tight
- Bound to the email requested — a token for `alice@example.com` cannot sign someone in as `bob@example.com` even if leaked

Explicitly not doing:

- No 6-digit code as an alternative.** Codes are more phishable than links, and the friction of typing them back into the app defeats the purpose. Link only.
- No "resend to a different email" option.** If the user typed the wrong address, they use "Start over" — resending to a different address on the same session is a phishing surface.

3.4 Post-verification name collection (magic link only)

If the magic-link user is new (`users.name IS NULL`), after ticket exchange the app renders:



Only this. No company, no role, no use case, no phone number, no password. Every one of those fields would add 5-10% drop-off (established SaaS metric) with no operational value.

- Name is 1-100 chars, trimmed, non-empty. Emoji allowed (people put them in their display names).
- Product updates checkbox **unchecked by default** — opt-in, not opt-out. CAN-SPAM compliant, GDPR-compliant, and the right thing to do.
- On Continue: PATCH `/v1/users/me` with `{ name, marketingEmailsOptIn }`, then transition to account onboarding (§5).

Google users skip this screen — Google's `given_name / family_name / name` claim populates `users.name` on provisioning, marketing opt-in defaults to `false`, and they go straight to account onboarding.

4. Tauri deep-link handling

4.1 Protocol registration

Register `jaroku://` as a custom URL scheme in Tauri's config:

```
# tauri.conf.json (excerpt)
"protocol": {
  "scheme": "jaroku",
  "description": "Jaroku deep link handler"
}
```

Handled by Tauri's built-in `tauri-plugin-deep-link`. Each OS has its own registration mechanism (Info.plist on macOS, registry on Windows, .desktop file on Linux) — the plugin handles this automatically at install time.

4.2 Deep-link paths

Two paths at launch:

- `jaroku://auth/complete?ticket=<ticket>` — auth callback (from §3.2 and §3.3)
- `jaroku://open?workspace=<id>&resource=<path>` — reserved for future in-app deep links (share a specific agent/run/trace); not implemented in this spec, but the URL space is claimed

4.3 App state when deep-link arrives

Three cases the handler must cover:

1. **App is running, in foreground.** Direct handoff to the deep-link handler. Simplest case.
2. **App is running, in background.** Bring window to front, then handle. macOS/Windows/Linux all support this via Tauri's window management.
3. **App is not running.** OS launches the app with the deep-link URL as a command-line argument. Tauri's plugin surfaces this identically to the running-app case, but the app must be sure to finish its own startup (§2) before processing the deep-link — a deep-link that arrives before first-run is complete should show first-run first, then process the auth callback after.

4.4 Single-use ticket exchange

The deep-link handler:

1. Extracts `ticket` from the URL
2. Calls `POST /v1/auth/session` with `{ ticket }` from the Tauri app
3. Backend verifies the ticket (signed, unexpired, unused, atomic consumption), returns a real session (a durable auth token stored securely on-device)

- 4. Session token is stored in OS-native secure storage: **macOS Keychain, Windows Credential Manager, Linux Secret Service**. Tauri has plugins for this. **Never localStorage, never a plain file on disk.**
- 5. WebSocket connection to the local backend is established with the session
- 6. If new user without a name (magic link): show \$3.4 screen
- 7. Otherwise: if onboarding_completed_at IS NULL, show account onboarding (\$5); else show the main app

4.5 Deep-link failure modes

Case	Behavior
Ticket expired (>60s)	Show "That link expired. Try signing in again." with a back-to-signin button. Do not silently retry — user needs to know why.
Ticket already used	Same message as expired. Do not distinguish; a used-ticket message is a fingerprinting signal for account existence.
Ticket signature invalid	Same message. Log a security event (auth.invalid_ticket) with the offending ticket for later analysis.
Deep-link arrives while first-run is incomplete	Queue it — complete first-run, then process on transition to sign-in.
Deep-link arrives while a different user is signed in	Prompt: "Sign in as [new user email]? This will sign out [current user]." with explicit Cancel and Continue. Never silent user swap.
Deep-link arrives with malformed URL	Ignore, log auth.malformed_deeplink . Do not error-toast the user — a browser or OS quirk should not surface as a scary message.

5. Account onboarding (per-user)

Runs when users.onboarding_completed_at IS NULL . Sequential, linear (Option A from the earlier decision). Each step has "Skip for now" available.

5.1 Screens

Step 1: Welcome (personalized)

Welcome, {firstName}

Let's get your workspace set up in about a minute.

We'll:

- Name your workspace
- Connect a model provider (optional)
- Generate your first agent (optional)

[Let's go →]

Skip setup

"Skip setup" is available. Clicking it lands the user in the main app with a **persistent top banner**: "Finish setting up your workspace →" that reopens onboarding.

Step 2: Workspace name

Name your workspace

{firstName}'s workspace

You can rename it later.

[Continue] [Skip for now]

Pre-filled with {firstName}'s workspace . User can edit or accept. Skip uses the pre-filled value silently — a workspace must exist for the user to do anything, so this step cannot truly be skipped in terms of workspace creation; skip just means "accept the default and move on."

- 1-60 chars, trimmed, non-empty
- Uniqueness not enforced across users (two users can both have "Personal workspace")

- On continue: `POST /v1/workspaces` with `{ name }` — creates the workspace, subscription defaults to Free (per subscription spec), user is added as owner

Step 3: Model provider (optional)

Connect a model provider

Jaroku uses your own API key to run models. Your key stays on your device.

☐ Anthropic (recommended)
☐ OpenAI
☐ Google

API key

sk-ant-...

[Where do I find this?]

✓ Validated

[Continue]
[Skip for now]

- Key is validated live on entry — same validation flow as the Secrets tab spec (\$5.3 of Secrets tab): make a cheapest-possible authenticated call (models-list), success → checkmark and store, failure → do not store, show provider error verbatim-but-sanitized.
- **Key is stored via SecretStore** (envelope encryption, no plaintext return path) — same infrastructure the Secrets tab uses. This step is effectively pre-populating the Secrets tab's Model Providers group.
- "Where do I find this?" opens a helpful documentation page per provider — links to Anthropic Console / OpenAI Platform / Google AI Studio.
- **Skip is genuinely first-class here.** A user who skips can still use the offline dry-run mode (v0.0.3) for their first agent, and can add a key later from the Secrets tab. The sample agent generated in Step 4 respects this.

Step 4: First agent (optional)

Generate your first agent

Pick a starting point:

☐ Weather + calculator (sample)
A simple 2-tool agent. Runs offline.
☐ Describe your own
Type what you want and Jaroku will generate it.
[Describe...]

[Generate]
[Skip for now]

- **Weather+calculator** is the existing v0.0.1 test agent, which runs offline (dry-run) with no API key. This is deliberately the default recommendation — it works for every user regardless of whether they set up a provider key.
- "Describe your own" opens an inline textarea and, on Generate, kicks off the normal agent generation flow (v0.0.3) — with the plan gate (v0.1.10) shown as a normal turn. The user sees their first plan card as part of onboarding, which is the "wow" moment.
- Skip lands the user in the main app with an empty state ("Your first agent will appear here — generate one from the composer"). No fake sample data.

Step 5: You're ready

✓

You're all set

A few things to try next:

- Run your agent to see it work
- Check the Graph tab to see its structure
- Ask Jaroku to change it in the composer

[Open Jaroku]

On "Open Jaroku": set `users.onboarding_completed_at = now()`, transition to the main app.

5.2 Definition of "onboarding completed"

`onboarding_completed_at` is set when:

- All mandatory steps completed (workspace name — the only truly mandatory step in this flow; provider key and first agent are optional)
- **AND** the user has taken an engagement action: either clicked "Open Jaroku" on Step 5, OR closed the onboarding modal via any means after reaching at least Step 3

Auth alone does not count. The engagement moment is what makes onboarding meaningful — a user who signed in and immediately closed the app has not been onboarded.

5.3 Resume-from-interruption

`users.onboarding_step` tracks the current step (integer 1-5). Updated after each step advances.

- User closes app mid-onboarding at step 3 → `onboarding_step = 3` , `onboarding_completed_at = NULL`
- User signs in next time → account onboarding shows, starts at step 3, not step 1
- Steps already completed are not re-done. Workspace already created → step 2 is skipped on resume.
- Data already saved (workspace, provider key) is not re-collected — it's re-shown for confirmation only if the user navigates back to that step.

5.4 "Restart onboarding" from settings

Available in `/settings/account`:

```
| Restart onboarding tour |
| Walk through the setup screens again. |
| Your workspace and settings won't change. |
| [ Restart ] |
```

Click → sets `onboarding_completed_at = NULL` , `onboarding_step = 1` , reloads app into onboarding. Existing workspace, keys, and agents remain intact — only the flag resets. Steps 2-4 pre-fill from existing state and read as "confirm or change" rather than "create."

6. Data model


```
-- migration 025 (verify actual next number before writing)

-- Extend users table with onboarding + auth metadata
ALTER TABLE users ADD COLUMN name text NULL;
ALTER TABLE users ADD COLUMN email_verified boolean NOT NULL DEFAULT false;
ALTER TABLE users ADD COLUMN auth_provider text NOT NULL CHECK (auth_provider IN ('google','magic_link'));
ALTER TABLE users ADD COLUMN marketing_emails_opt_in boolean NOT NULL DEFAULT false;
ALTER TABLE users ADD COLUMN onboarding_started_at timestampz NULL;
ALTER TABLE users ADD COLUMN onboarding_completed_at timestampz NULL;
ALTER TABLE users ADD COLUMN onboarding_step smallint NOT NULL DEFAULT 1;

-- Magic link tokens (hashed, never store raw)
magic_link_tokens(
  token_hash text PRIMARY KEY,      -- sha256 of the raw token
  email text NOT NULL,
  ip_address inet NOT NULL,         -- for rate limiting and abuse tracking
  user_agent text NOT NULL,
  created_at timestampz NOT NULL DEFAULT now(),
  expires_at timestampz NOT NULL,
  consumed_at timestampz NULL,
  INDEX (email, created_at)        -- for rate limit lookups
)

-- OAuth state tokens (signed, single-use, short-lived)
oauth_state_tokens(
  state text PRIMARY KEY,           -- the signed state value
  provider text NOT NULL CHECK (provider IN ('google')),
  code_verifier text NOT NULL,      -- PKCE verifier, stored to complete exchange
  tauri_nonce text NOT NULL,        -- binds this state to the Tauri instance
  created_at timestampz NOT NULL DEFAULT now(),
  expires_at timestampz NOT NULL,
  consumed_at timestampz NULL
)

-- Session tickets (60-second window between web callback and Tauri app exchange)
session_tickets(
  ticket_hash text PRIMARY KEY,     -- sha256 of the raw ticket
  user_id uuid NOT NULL,
  provider text NOT NULL,
  created_at timestampz NOT NULL DEFAULT now(),
  expires_at timestampz NOT NULL,
  consumed_at timestampz NULL
)

-- Rate limits for magic link requests
magic_link_rate_limits(
  key text PRIMARY KEY,             -- 'email:<addr>' or 'ip:<ip>'
  window_start timestampz NOT NULL,
  count integer NOT NULL DEFAULT 0
)
```

Never store raw tokens — always hashes. `token_hash` uses SHA-256 (fast, collision-resistant enough for this purpose — this is not password storage where Argon2id is required, this is short-lived opaque token storage where the security property is "database dump does not enable use of unconsumed tokens").

RLS is not applicable to `magic_link_tokens`, `oauth_state_tokens`, `session_tickets`, or `magic_link_rate_limits` — these are pre-authentication tables with no tenant scope. But they must be:

- Scoped to server-side access only (never exposed to any client)
- Cleaned up by a scheduled job that deletes expired-or-consumed rows daily

7. Backend endpoints

All new endpoints. Existing `/v1/auth/session` from v0.2.6 is extended to accept a ticket in addition to whatever it accepts today.

Method	Route	Auth	Purpose
POST	/v1/auth/oauth/google/start	none	Generate state + PKCE, return authorization URL
GET	/oauth/google/callback	none	Handle Google's redirect, exchange code, mint ticket, redirect to jaroku://
POST	/v1/auth/magic-link	none	Rate-limit, generate token, send email, return 200 always
GET	/magic	none	Handle magic-link click, mint ticket, redirect to jaroku://
POST	/v1/auth/session	ticket	Exchange ticket for session (existing endpoint, extended)
POST	/v1/auth/signout	session	Invalidate session, clear WS
PATCH	/v1/users/me	session	Update name, marketing opt-in
POST	/v1/users/me/onboarding/step	session	Advance onboarding step (idempotent)
POST	/v1/users/me/onboarding/complete	session	Mark onboarding complete
POST	/v1/users/me/onboarding/restart	session	Reset onboarding flag for restart-tour

Rules that apply to every endpoint:

1. All non-auth routes require a valid session, checked in middleware (existing pattern from v0.2.6).
2. Anything referencing another user's data returns 404, not 403 (existing convention).
3. Rate-limit POST /v1/auth/magic-link at the IP and email level; both must apply.
4. Never log the raw token, ticket, magic link URL, or authorization code — only their hashes and metadata.
5. Every auth event writes to audit_log: auth.oauth_started · auth.oauth_completed · auth.magic_link_sent · auth.magic_link_consumed · auth.session_created · auth.session_invalidated · auth.invalid_ticket · auth.rate_limited . auth.rate_limited and auth.invalid_ticket are the highest-signal rows — do not sample.

8. Email delivery

Magic link emails must actually arrive, in the primary inbox, on the first try. This is not a place to cut corners — an email that lands in spam is an unusable sign-in flow.

8.1 Provider

Use **Resend** or **Postmark** for transactional email. Both are designed for this exact use case (authentication emails), have high deliverability, and give per-message tracking. Do not use SendGrid Free (deliverability is bad for auth email) or a generic SMTP relay.

Store the API key in SecretStore — same discipline as every other credential.

8.2 Domain setup (production-blocking)

Before launch, the sending domain (auth.jaroku.dev or mail.jaroku.dev) must have:

- **SPF record** authorizing the email provider
- **DKIM record** with the provider's public key
- **DMARC record** with p=quarantine at minimum (start with p=none for a week to observe, then tighten)

Without these, emails go straight to spam at Gmail (which is 60%+ of the target audience). This is not optional.

8.3 Email content

Plain, minimal, transactional:

Subject: Sign in to Jaroku

Body (both HTML and plain text — always send both):

```

Sign in to Jaroku

Click the button below to sign in. This link expires in 15 minutes.

[ Sign in to Jaroku ] (button linking to https://auth.jaroku.dev/magic?token=...)

If you didn't request this, you can safely ignore this email.

—
Jaroku
{unsubscribe/preferences link, ONLY for marketing emails – NOT for auth emails}

```

Explicitly:

- No marketing copy in the auth email. It's transactional.
- No unsubscribe link on transactional emails (they're not subject to CAN-SPAM's unsubscribe requirement, and adding one weakens

the transactional classification with providers)

- Sender name: `Jaroku`
- Sender address: `sign-in@auth.jaroku.dev` (or similar) — a real, monitored address, not `noreply@`
- Plain-text version is a literal fallback, not a "click here for HTML version" — it contains the same link and message

8.4 Delivery failure handling

- Resend/Postmark webhooks report bounces and complaints. Handle both:
- **Hard bounce** (invalid email): mark the email as undeliverable, block future magic links to this address, surface in-app on next attempt ("This email address isn't reachable. Try another.")
- **Complaint** (marked as spam): block future magic links to this address, log for review — this is a signal something went wrong
- Soft bounces (temporary): allow retry after 5 minutes
- Never surface delivery failures to a different user (email enumeration risk)

9. Resume-from-interruption logic

This applies to both first-run and account onboarding. The general principle: **any onboarding step that saved state is not repeated on resume.**

9.1 First-run resume

- If `~/ .jaroku/app-initialized` marker exists → skip first-run entirely
- If marker does not exist but partial state exists (`~/ .jaroku/venv/` half-created) → offer "Continue setup" that verifies each step and re-runs any incomplete ones
- If everything is broken (config file corrupt, marker present but venv missing) → offer "Reset device setup" that wipes `~/ .jaroku/` and re-runs first-run from scratch

9.2 Account onboarding resume

- On sign-in, check `users.onboarding_completed_at`. If not null, no onboarding — go straight to main app.
- If null and `onboarding_step > 1`, resume at that step. Show a small "Welcome back — let's finish setting up" hint at the top of the step's screen.
- If null and `onboarding_step = 1`, show the full onboarding from Step 1.

9.3 Explicit skip vs interruption

These are different, and the flag distinguishes them:

- **Skip** (clicked "Skip for now"): advances `onboarding_step`, sets `onboarding_completed_at` only if all subsequent steps also skipped or completed
- **Interruption** (closed app, killed process): does not advance `onboarding_step`, resume shows the same step

10. Edge cases and failure modes

Case	Behavior
User signs in with Google, later signs in with magic link on same email	Same account (matched by verified email); auth provider on <code>users</code> reflects most recent. Never two accounts for one email.
User's Google account uses a different capitalization than their magic link	Emails are case-insensitive per RFC — normalize to lowercase before comparison; store as user entered but match as lowercase.
Magic link clicked 3 times in quick succession (browser preview + user click + prefetcher)	Atomic consumption ensures exactly one succeeds. Others get "That link expired or was already used."
User clicks magic link on a different device than they started on	Works — the deep-link opens the Jaroku app on that device. User signs in on the new device. This is a feature (cross-device sign-in), not a bug.
User clicks magic link, Jaroku isn't installed on that device	Deep-link fails at OS level. Web callback page detects this via a short JS timer and shows: "Install Jaroku, then return to your email."
User completes OAuth in browser but closes the browser before deep-link fires	State token is single-use — user must restart. Show "Sign-in didn't complete. Try again." on next launch.
Email provider is down when magic link is requested	Return an actionable error to the user: "Couldn't send email right now. Try again in a minute, or use Google sign-in." Do not silently fail.
User's <code>~/ .jaroku/</code> is on a network drive that becomes unavailable mid-session	Detect on next disk write attempt; show a clear "Storage unavailable" screen with retry, do not corrupt state
Multiple Tauri app instances launched (user double-clicks the icon a lot)	Tauri's <code>single-instance</code> plugin — second launch focuses the first window, does not start a second process. Register a deep-link received on the second launch to be routed to the first.
Onboarding completed but user was created with <code>name: NULL</code>	Should not happen (name is required post-auth for magic-link, comes from Google for OAuth) — but if it does, <code>PATCH /v1/users/me</code> requires a non-null name on any subsequent update, so this self-heals on first settings edit.
User restarts onboarding but their workspace was deleted in between	Step 2 pre-fill fails → treated as "create new" instead of "confirm existing" — never crashes

11. Build order

M1 — Tauri deep-link handling. Protocol registration, plugin integration, three-app-state handling (running/background/not-running). Test with a hand-crafted `jaroku://` URL, no backend involvement.

M2 — First-run flow. Screens 1-3, runtime dependency install via `uv`, checkpoint DB init, marker file logic, offline handling. No auth yet.

M3 — Google OAuth (backend). State token generation with PKCE, `/oauth/google/callback` endpoint, Google client configuration in dashboard, ticket minting. Test end-to-end against a real Google OAuth app in a test Google account.

M4 — Google OAuth (client). "Continue with Google" button, open-system-browser, deep-link callback handler, ticket exchange, session storage in OS keychain.

M5 — Magic link (backend). `POST /v1/auth/magic-link`, token generation, rate limiting, email sending via Resend/Postmark, `/magic` GET endpoint, ticket minting. Domain + SPF/DKIM/DMARC setup done in parallel — must complete before launch.

M6 — Magic link (client). Email entry form, "check your email" confirmation screen, resend logic, deep-link callback (reused from M4).

M7 — Post-verification name collection. `PATCH /v1/users/me`, the name-collection screen for magic-link users only.

M8 — Account onboarding. All 5 screens, `onboarding_step` tracking, skip logic, resume logic, restart-from-settings.

M9 — Provider key onboarding. Integration with the Secrets tab's Model Providers flow — pre-populate from onboarding step 3.

M10 — First agent generation. Weather+calculator sample flow, "describe your own" flow using existing generation infrastructure.

12. Acceptance criteria

First-run

1. First-run runs exactly once per device install, gated by the marker file.
2. Wiping `~/ .jaroku/` re-triggers first-run on next launch.
3. Runtime install failure surfaces a clear error with retry, get-help, and quit options — never a silent failure.
4. Offline first-run completes all local steps and fails cleanly at the network-required step.

Google OAuth

5. OAuth flow uses PKCE with `S256` — verified by inspecting the actual authorization URL.

6. State token is single-use — a replayed callback returns an error.
7. New Google users are provisioned with `name` from the ID token's claim, `email_verified: true`, `auth_provider: 'google'`.
8. Scope requested is exactly `openid email profile` — no more.
9. Session ticket is exchanged within 60 seconds or expires cleanly.

Magic link

0. `POST /v1/auth/magic-link` returns 200 regardless of whether the email exists — verified by comparing responses for a known and unknown email.
1. Rate limit blocks a 4th request from the same email in an hour, and an 11th from the same IP in an hour.
2. Magic link token is stored hashed — the raw token cannot be reconstructed from the database.
3. Consumed token cannot be re-used — atomic consumption verified with a concurrent double-click test.
4. Expired token (>15 minutes) is refused with the same message as an already-consumed one.
5. Bounced email addresses are blocked from receiving further magic links.
6. Auth emails are sent from a domain with valid SPF, DKIM, and DMARC records, verified by checking inbox headers.

Deep-link

7. Deep-link works when the app is running, backgrounded, or not launched.
8. Only one Tauri instance runs — a second launch focuses the first.
9. A deep-link arriving mid-first-run is queued until first-run completes.
0. A deep-link for a different user than the current session prompts before swapping.

Session

1. Session token is stored in the OS-native secure store (Keychain / Credential Manager / Secret Service), never in `localStorage` or a plain file — verified by inspecting on-disk artifacts after sign-in.
2. `POST /v1/auth/signout` invalidates the session server-side, and the WS connection drops.

Account onboarding

3. `onboarding_completed_at` is set only when the user takes an engagement action past step 3, not merely on auth.
4. Interruption resumes at the interrupted step, not from step 1.
5. Skipped optional steps do not gate `onboarding_completed_at` — a user who skips provider key and first agent still lands in the main app with a completed flag.
6. Restart-from-settings preserves existing workspace, keys, and agents; only the flag resets.
7. The persistent "finish setup" banner appears when a user skips onboarding entirely.

Data safety

8. No raw magic link token, OAuth state, or session ticket is present in any log, error message, or telemetry payload — verified by grep across log output.
9. Email addresses are compared case-insensitively but stored as entered.
0. A user existing under one auth provider cannot be silently duplicated by signing in under another — the email match resolves to the same account.

13. Out of scope

- **GitHub, Microsoft, Apple, or Facebook OAuth.** Google + magic link covers 95%+; adding more creates choice fatigue.
- **SSO / SAML.** Enterprise territory, not this spec.
- **Two-factor authentication (TOTP, SMS).** Magic link is already a proof-of-email-possession factor; adding a second factor is a legitimate future addition but launch-blocking scope creep.
- **Passwords.** Deliberately excluded — magic link is the choice.
- **Passwordless code (6-digit).** Rejected in favor of link-only.
- **Social login as sole identifier** (users identified only by their Google `sub`, no email). Email is always the primary identifier; Google's `sub` is a secondary lookup used to link, never to identify alone.
- **Account recovery flow for a lost email.** Genuine problem, but requires manual support handling at this scale — automate later.
- **Custom onboarding per referral source** (different flow for HN vs Twitter). Interesting for growth, not launch-blocking.
- **Video product tour.** Rejected — good empty states in the main app do this job better.
- **A/B testing framework for onboarding.** Premature — you need users first, then you optimize.